

n-step Bootstrapping: Reinforcement Learning - Lecture 8

Felipe Meneguzzi
`felipe.meneguzzi@pucrs.br`

PUCRS

Material adapted from:
Martha and Adam White - University of Alberta/Coursera
Sutton and Barto - Reinforcement Learning: An Introduction (2nd ed.)
David Silver - Reinforcement Learning

Outline

- 1 Recap
- 2 n -step Bootstrapping
 - n -step TD Prediction
 - n -step Sarsa
- 3 Eligibility Traces and λ Algorithms
 - TD(λ)
 - Sarsa(λ)
- 4 From DP to MC and TD

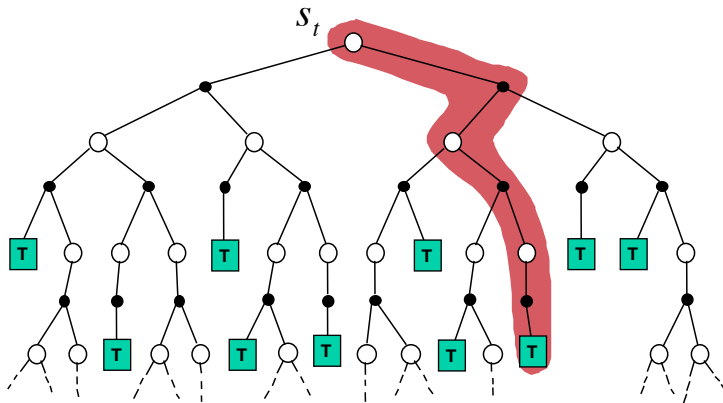
- Last lecture:
 - Control
 - Comparison with Monte Carlo methods
- This lecture (Sutton and Barto 7.1-7.3 and 12.1, 12.2, and 12.7)
 - n-Step **Bootstrapping** and λ updates
 - Eligibility Traces
- Next lecture
 - Integrating Planning and Learning
 - Dyna-Q

Advantages and Disadvantages of MC vs. TD

- TD exploits Markov property
 - Usually more efficient in Markov environments
- MC does not exploit Markov property
 - Usually more effective in non-Markov environments

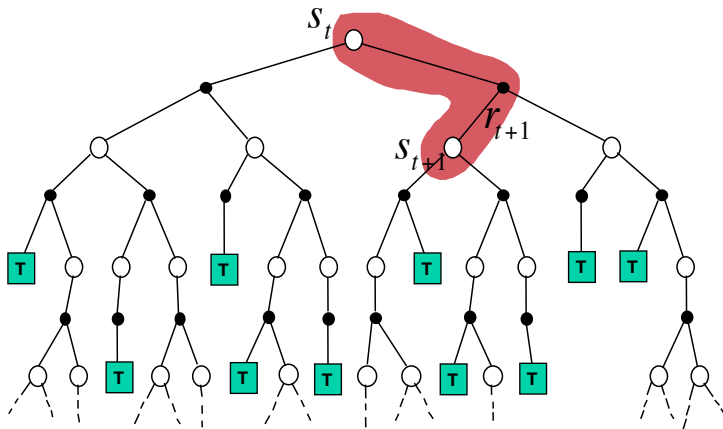
Unified View: Monte-Carlo Backup

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$



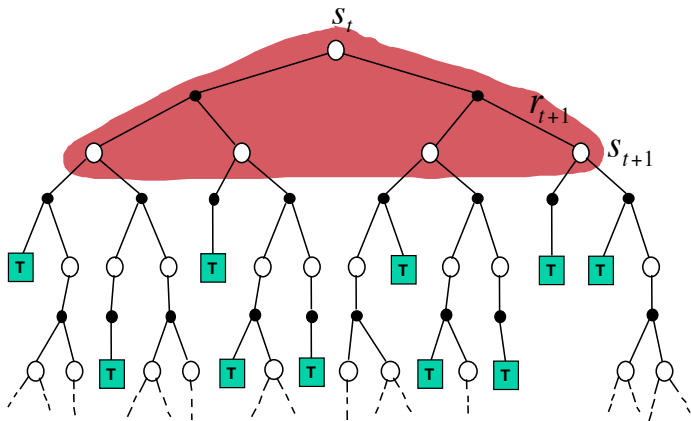
Unified View: Temporal-Difference Backup

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} - \gamma V(S_{t+1}) - V(S_t))$$



Unified View: Dynamic Programming Backup

$$V(S_t) \leftarrow \mathbb{E} [R_{t+1} + \gamma V(S_{t+1})]$$



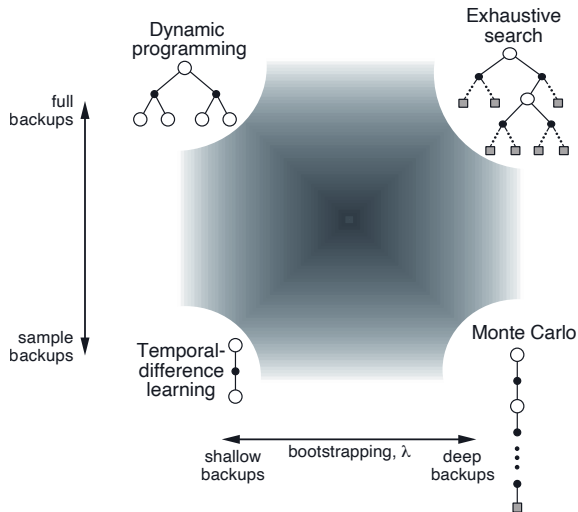
Bootstrapping and Sampling

- **Bootstrapping:** update involves an estimate
 - MC does not bootstrap
 - DP bootstraps
 - TD bootstraps
- **Sampling:** update samples an expectation
 - MC samples
 - DP does not sample
 - TD samples

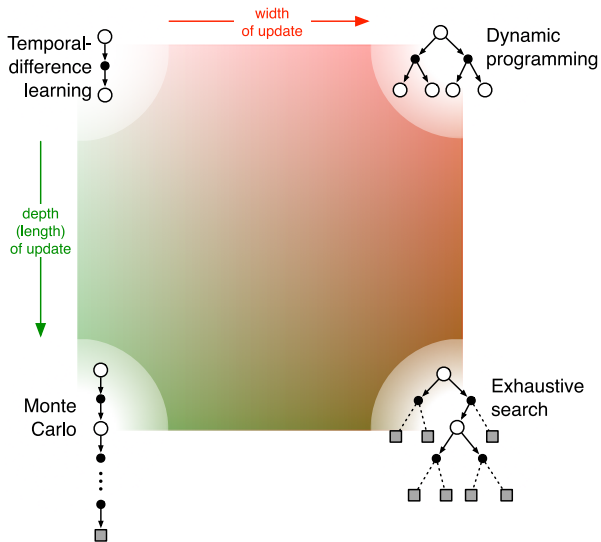
Unifying Monte Carlo and Temporal Difference

- Monte Carlo (MC) performs deep backups
- Temporal Difference (TD) performs shallow backups
- Can we reach a compromise to achieve a balance between:
 - Immediate feedback from new knowledge
 - Meaningful updates from significant state changes
- In this lecture, we look at a class of n-step bootstrapping methods, and their generalization
 - n-step TD
 - Eligibility Traces
 - λ algorithms

Unified View of Reinforcement Learning



Unified View of Reinforcement Learning

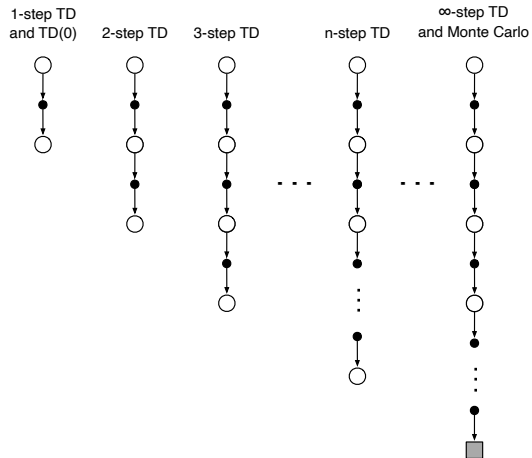


Outline

- 1 Recap
- 2 *n*-step Bootstrapping
 - n*-step TD Prediction
 - n*-step Sarsa
- 3 Eligibility Traces and λ Algorithms
 - TD(λ)
 - Sarsa(λ)
- 4 From DP to MC and TD

n -Step Prediction

- Let TD target look n steps into the future



n -Step Return

- Consider the following n -step returns for $n = 1, 2, \infty$:

$$n = 1 \text{ (TD)} \quad G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$$

$$n = 2 \quad G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$$

$$\vdots \quad \quad \quad \vdots$$

$$n = \infty \text{ (MC)} \quad G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-1} R_T$$

- Define the n -step return

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- n -step temporal-difference learning

$$V(S_t) \leftarrow V(S_t) + \alpha \left(G_t^{(n)} - V(S_t) \right)$$

n -step TD Prediction

n -step TD for estimating $V \approx v_\pi$

Require: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

Require: All store and access operations (for S_t and R_t) can take their index mod $n + 1$

```
1: function nStepTDPrediction( $\alpha, \pi, n$ )
2:   Initialize  $V(s) \in \mathbb{R}$  arbitrarily for all  $s \in \mathcal{S}^+$ 
3:   loop for each episode
4:     Initialize and store  $S_0 \neq \text{terminal}$ 
5:      $T \leftarrow \infty$ 
6:     loop for  $t = 0, 1, 2, \dots$ 
7:       if  $t < T$  then
8:         Take an action according  $\pi(\cdot \mid S_t)$ 
9:         Observe and store the next reward as  $R_{t+1}$  and state as  $S_{t+1}$ 
10:        if  $S_{t+1}$  is terminal then  $T \leftarrow t + 1$ 
11:         $\tau \leftarrow t - n + 1$   $\triangleright \tau$  is the time whose state's estimate is being updated
12:        if  $\tau \geq 0$  then
13:           $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$ 
14:          if  $\tau + n < T$  then  $G \leftarrow G + \gamma^n V(S_{\tau+1})$   $\triangleright G_{\tau:\tau+n}$ 
15:           $V(S_\tau) \leftarrow V(S_\tau) + \gamma [G - V(S_\tau)]$ 
until  $\tau = T - 1$ 
```

Driving Home Example (2-step TD)

leave



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

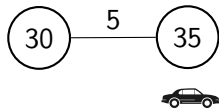
$$\tau \leftarrow t - n + 1$$

$$\alpha = 1 \quad \gamma = 1$$

Driving Home Example (2-step TD)

leave

exit



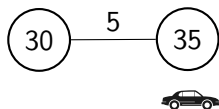
$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$
$$\tau \leftarrow t - n + 1$$

$$\alpha = 1 \quad \gamma = 1$$

Driving Home Example (2-step TD)

leave

exit



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

$$\tau \leftarrow t - n + 1$$

$$\tau \leftarrow 0 - 2 + 1 = -1$$

No update

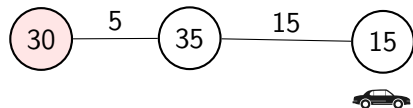
$$\alpha = 1 \quad \gamma = 1$$

Driving Home Example (2-step TD)

leave

exit

exit highway



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

$$\tau \leftarrow t - n + 1$$

$$\tau \leftarrow 1 - 2 + 1 = 0$$

$$V(\text{leave}) \leftarrow V(\text{leave}) + \alpha (R_1 + \gamma R_2 + \gamma^2 V(\text{exit highway}) - V(\text{leave}))$$

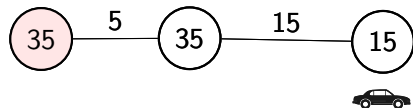
$$\alpha = 1 \quad \gamma = 1$$

Driving Home Example (2-step TD)

leave

exit

exit highway



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

$$\tau \leftarrow t - n + 1$$

$$\tau \leftarrow 1 - 2 + 1 = 0$$

$$V(\text{leave}) \leftarrow V(\text{leave}) + \alpha (R_1 + \gamma R_2 + \gamma^2 V(\text{exit highway}) - V(\text{leave}))$$

$$\leftarrow 30 + (5 + 15 + 15 - 30)$$

$$\alpha = 1 \quad \gamma = 1$$

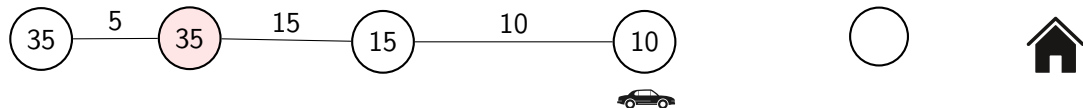
Driving Home Example (2-step TD)

leave

exit

exit highway

secondary road



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

$$\tau \leftarrow t - n + 1$$

$$\tau \leftarrow 2 - 2 + 1 = 1$$

$$V(\text{exit}) \leftarrow V(\text{exit}) + \alpha (R_2 + \gamma R_3 + \gamma^2 V(\text{secondary road}) - V(\text{exit}))$$

$$\alpha = 1 \quad \gamma = 1$$

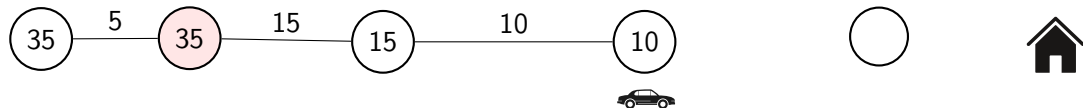
Driving Home Example (2-step TD)

leave

exit

exit highway

secondary road



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

$$\tau \leftarrow t - n + 1$$

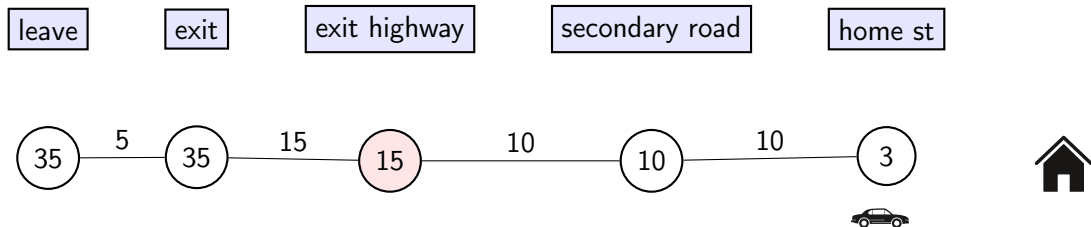
$$\tau \leftarrow 2 - 2 + 1 = 1$$

$$V(\text{exit}) \leftarrow V(\text{exit}) + \alpha (R_2 + \gamma R_3 + \gamma^2 V(\text{secondary road}) - V(\text{exit}))$$

$$\leftarrow 35 + (15 + 10 + 10 - 35)$$

$$\alpha = 1 \quad \gamma = 1$$

Driving Home Example (2-step TD)



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

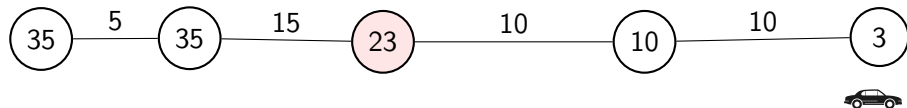
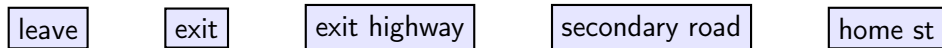
$$\tau \leftarrow t - n + 1$$

$$\tau \leftarrow 3 - 2 + 1 = 2$$

$$V(\text{exit highway}) \leftarrow V(\text{exit highway}) + \alpha (R_3 + \gamma R_4 + \gamma^2 V(\text{home st}) - V(\text{exit highway}))$$

$$\alpha = 1 \quad \gamma = 1$$

Driving Home Example (2-step TD)



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

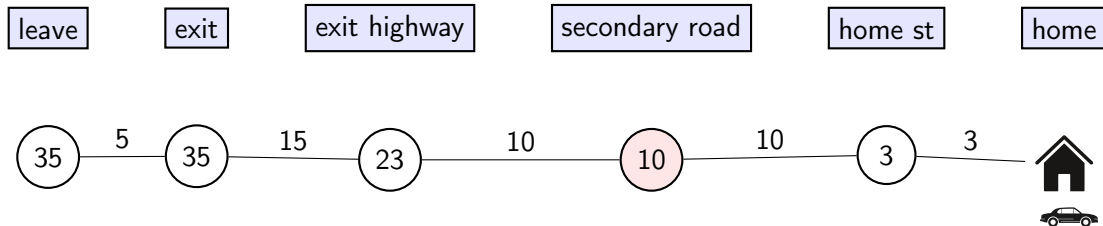
$$\tau \leftarrow t - n + 1$$

$$\tau \leftarrow 3 - 2 + 1 = 2$$

$$\begin{aligned} V(\text{exit highway}) &\leftarrow V(\text{exit highway}) + \alpha (R_3 + \gamma R_4 + \gamma^2 V(\text{home st}) - V(\text{exit highway})) \\ &\leftarrow 15 + (10 + 10 + 3 - 15) \end{aligned}$$

$$\alpha = 1 \quad \gamma = 1$$

Driving Home Example (2-step TD)



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

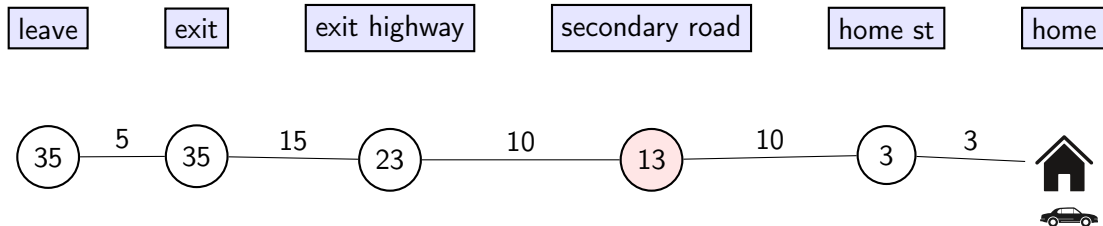
$$\tau \leftarrow t - n + 1$$

$$\tau \leftarrow 4 - 2 + 1 = 3$$

$$V(\text{secondary road}) \leftarrow V(\text{secondary road}) + \alpha (R_4 + \gamma R_5 + \gamma^2 V(\text{home}) - V(\text{secondary road}))$$

$$\alpha = 1 \quad \gamma = 1$$

Driving Home Example (2-step TD)



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

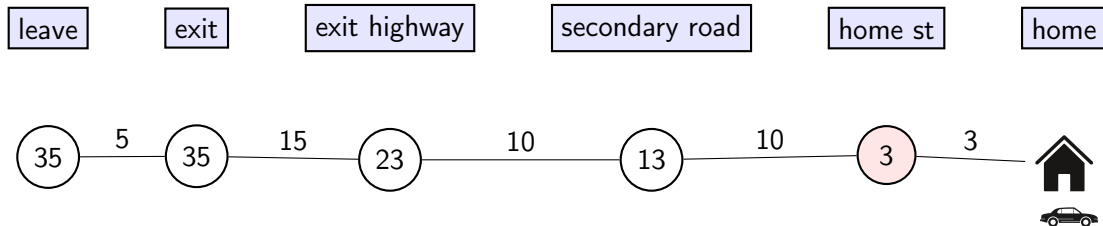
$$\tau \leftarrow t - n + 1$$

$$\tau \leftarrow 4 - 2 + 1 = 3$$

$$\begin{aligned} V(\text{secondary road}) &\leftarrow V(\text{secondary road}) + \alpha (R_4 + \gamma R_5 + \gamma^2 V(\text{home}) - V(\text{secondary road})) \\ &\leftarrow 10 + (10 + 3 + 0 - 10) \end{aligned}$$

$$\alpha = 1 \quad \gamma = 1$$

Driving Home Example (2-step TD)



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

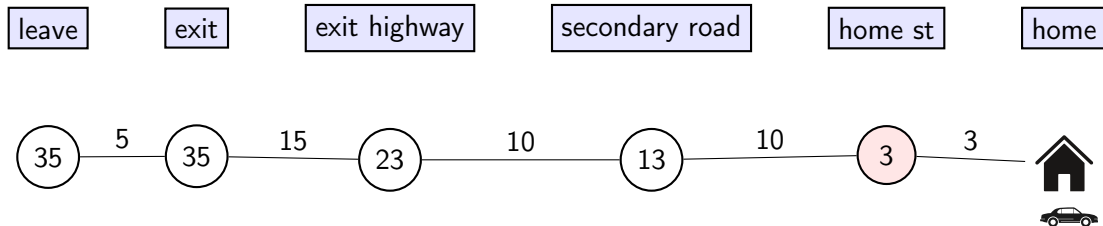
$$\tau \leftarrow t - n + 1$$

$$\tau \leftarrow 4 - 2 + 1 = 3$$

$$V(\text{home st}) \leftarrow V(\text{home st}) + \alpha (R_5)$$

$$\alpha = 1 \quad \gamma = 1$$

Driving Home Example (2-step TD)



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$

$$\tau \leftarrow t - n + 1$$

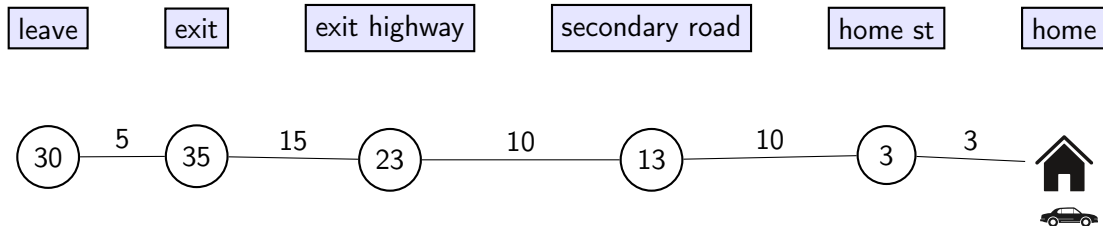
$$\tau \leftarrow 4 - 2 + 1 = 3$$

$$V(\text{home st}) \leftarrow V(\text{home st}) + \alpha (R_5)$$

$$\leftarrow 10 + (3 - 3)$$

$$\alpha = 1 \quad \gamma = 1$$

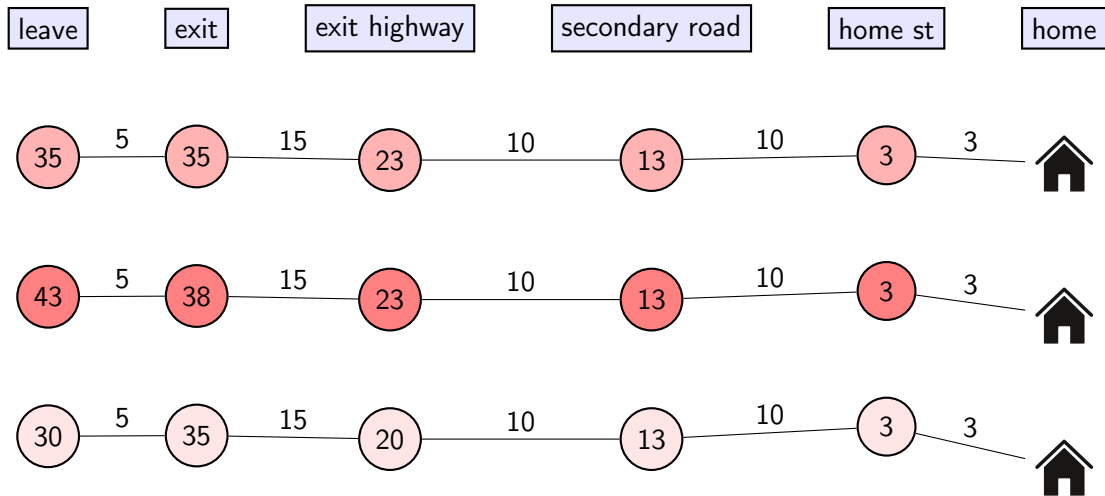
Driving Home Example (2-step TD)



$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t))$$
$$\tau \leftarrow t - n + 1$$

$$\alpha = 1 \quad \gamma = 1$$

Comparison of n-step vs. MC vs. TD



n -Step Sarsa Backups

- Let Sarsa target look n steps into the future



- Much like TD n -step returns but starts with actions and end in actions
- Switch states for actions (state-action pairs)
- Use ϵ -greedy policy

n-Step Sarsa

- Consider the following n-step returns for $n = 1, 2, \infty$:

$$n = 1 \text{ (Sarsa)} \quad q_t^{(1)} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

$$n = 2 \quad q_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 Q(S_{t+2}, A_{t+1})$$

$$\vdots$$

$$n = \infty \text{ (MC)} \quad q_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-1} R_T$$

- Define the n -step Q -return

$$q_t^{(n)} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n})$$

- n -step Sarsa updates $Q(s, a)$ towards n -step Q -return

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(q_t^{(n)} - Q(S_t, A_t) \right)$$

n -step Sarsa

n -step Sarsa for estimating $Q \approx q_*$ or q_π

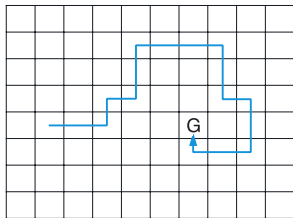
Require: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

Require: All store and access operations (for S_t , A_t and R_t) can take their index mod $n + 1$

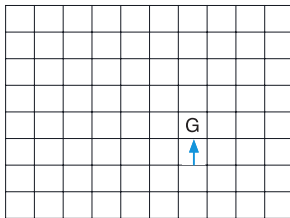
```
1: function nStepSarsa( $\alpha, \epsilon, \pi, n$ )
2:   Initialize  $Q(s, a) \in \mathbb{R}$  arbitrarily, for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$ 
3:   loop for each episode
4:     Initialize and store  $S_0 \neq \text{terminal}$ 
5:     Select and store an action  $A_0 \sim \pi(\cdot \mid S_0)$ 
6:      $T \leftarrow \infty$ 
7:     loop for  $t = 0, 1, 2, \dots$ 
8:       if  $t < T$  then
9:         Take an action  $A_t$ 
10:        Observe and store the next reward as  $R_{t+1}$  and state as  $S_{t+1}$ 
11:        if  $S_{t+1}$  is terminal then  $T \leftarrow t + 1$ 
12:        else Select and store an action  $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$ 
13:         $\tau \leftarrow t - n + 1$   $\triangleright \tau$  is the time whose state's estimate is being updated
14:        if  $\tau \geq 0$  then
15:           $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$ 
16:          if  $\tau + n < T$  then  $G \leftarrow G + \gamma^n Q(S_{\tau+1}, A_{\tau+1})$   $\triangleright G_{\tau:\tau+n}$ 
17:           $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \gamma [G - Q(S_\tau, A_\tau)]$ 
18:           $\triangleright$  If  $\pi$  is being learned, ensure  $\pi(\cdot \mid S_t)$  is  $\epsilon$ -greedy wrt  $Q$ 
        until  $\tau = T - 1$ 
```

n-step Sarsa Gridworld Example

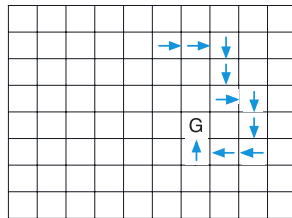
Path taken



Action values increased
by one-step Sarsa



Action values increased
by 10-step Sarsa



n -step Expected Sarsa

n -step Sarsa for estimating $Q \approx q_*$ or q_π

Require: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

Require: All store and access operations (for S_t , A_t and R_t) can take their index mod $n + 1$

```
1: function nStepSarsa( $\alpha, \epsilon, \pi, n$ )
2:   Initialize  $Q(s, a) \in \mathbb{R}$  arbitrarily, for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$ 
3:   loop for each episode
4:     Initialize and store  $S_0 \neq \text{terminal}$ 
5:     Select and store an action  $A_0 \sim \pi(\cdot \mid S_0)$ 
6:      $T \leftarrow \infty$ 
7:     loop for  $t = 0, 1, 2, \dots$ 
8:       if  $t < T$  then
9:         Take an action  $A_t$ 
10:        Observe and store the next reward as  $R_{t+1}$  and state as  $S_{t+1}$ 
11:        if  $S_{t+1}$  is terminal then  $T \leftarrow t + 1$ 
12:        else Select and store an action  $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$ 
13:         $\tau \leftarrow t - n + 1$   $\triangleright \tau$  is the time whose state's estimate is being updated
14:        if  $\tau \geq 0$  then
15:           $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$ 
16:          if  $\tau + n < T$  then  $G \leftarrow G + \gamma^n Q(S_{\tau+1}, A_{\tau+1})$   $\triangleright G_{\tau:\tau+n}$ 
17:           $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \gamma [G - Q(S_\tau, A_\tau)]$ 
18:           $\triangleright$  If  $\pi$  is being learned, ensure  $\pi(\cdot \mid S_t)$  is  $\epsilon$ -greedy wrt  $Q$ 
        until  $\tau = T - 1$ 
```

n -step Expected Sarsa

n -step Sarsa for estimating $Q \approx q_*$ or q_π

Require: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

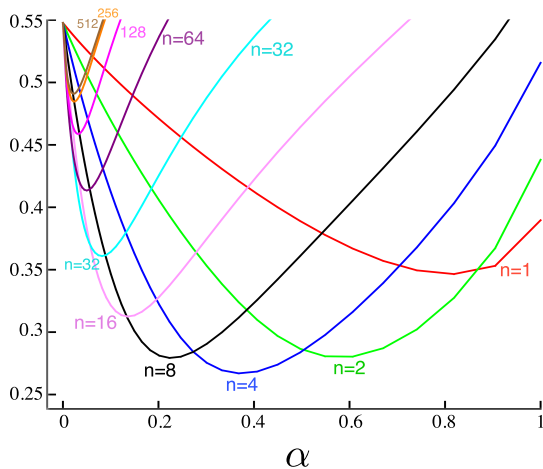
Require: All store and access operations (for S_t , A_t and R_t) can take their index mod $n + 1$

```
1: function nStepSarsa( $\alpha, \epsilon, \pi, n$ )
2:   Initialize  $Q(s, a) \in \mathbb{R}$  arbitrarily, for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$ 
3:   loop for each episode
4:     Initialize and store  $S_0 \neq \text{terminal}$ 
5:     Select and store an action  $A_0 \sim \pi(\cdot \mid S_0)$ 
6:      $T \leftarrow \infty$ 
7:     loop for  $t = 0, 1, 2, \dots$ 
8:       if  $t < T$  then
9:         Take an action  $A_t$ 
10:        Observe and store the next reward as  $R_{t+1}$  and state as  $S_{t+1}$ 
11:        if  $S_{t+1}$  is terminal then  $T \leftarrow t + 1$ 
12:        else Select and store an action  $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$ 
13:         $\tau \leftarrow t - n + 1$   $\triangleright \tau$  is the time whose state's estimate is being updated
14:        if  $\tau \geq 0$  then
15:           $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$ 
16:          if  $\tau + n < T$  then  $G \leftarrow G + \gamma^n \sum_a \pi(a \mid S_{\tau+1}) Q(S_{\tau+1}, a)$   $\triangleright G_{\tau:\tau+n}$ 
17:           $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \gamma [G - Q(S_\tau, A_\tau)]$ 
18:           $\triangleright$  If  $\pi$  is being learned, ensure  $\pi(\cdot \mid S_t)$  is  $\epsilon$ -greedy wrt  $Q$ 
until  $\tau = T - 1$ 
```

Large Random Walk Example

MRP with 19 states, -1 outcome on the left, 10 episodes.

Average
RMS error
over 19 states
and first 10
episodes



Outline

- 1 Recap
- 2 *n*-step Bootstrapping
 - n*-step TD Prediction
 - n*-step Sarsa
- 3 Eligibility Traces and λ Algorithms
 - TD(λ)
 - Sarsa(λ)
- 4 From DP to MC and TD

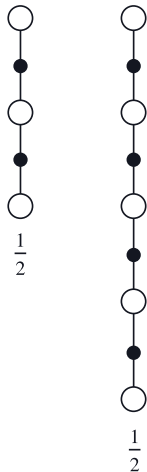
Averaging n -Step Returns

- We can average n -step returns over different n
e.g. average the 2-step and 4-step returns

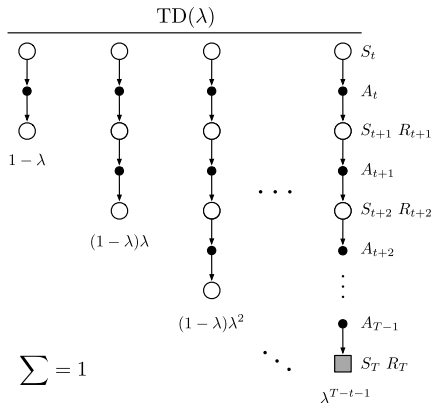
$$\frac{1}{2} + G^2 + \frac{1}{2} + G^4$$

- Combines information from two different time-steps
- Can we efficiently combine information from all time-steps?

One backup



λ -return



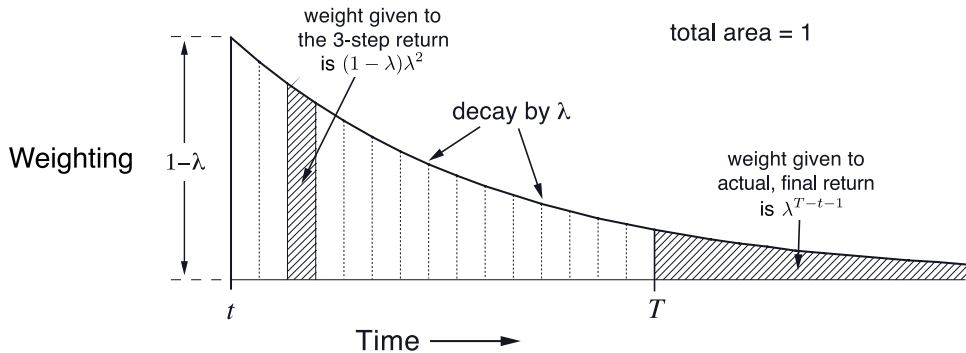
- The λ -return G_t^λ combines all n -step returns $G_t^{(n)}$
- Using weight $(1 - \lambda)\lambda^{n-1}$

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- Forward-view TD(λ)

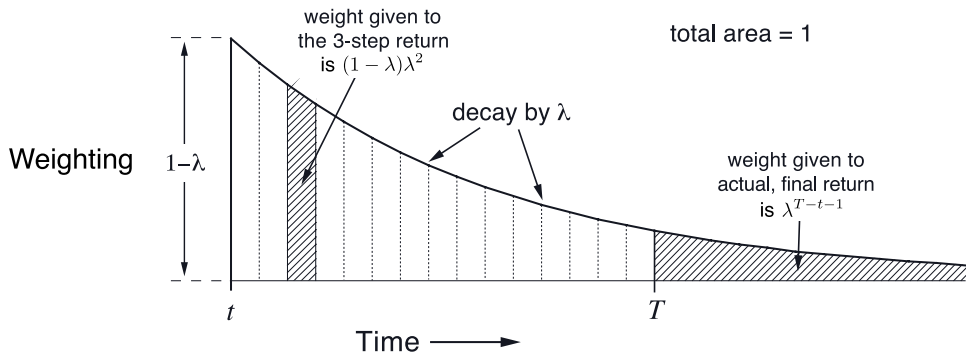
$$V(S_t) \leftarrow V(S_t) + \alpha (G_t^\lambda - V(S_t))$$

TD(λ) Weighing Function



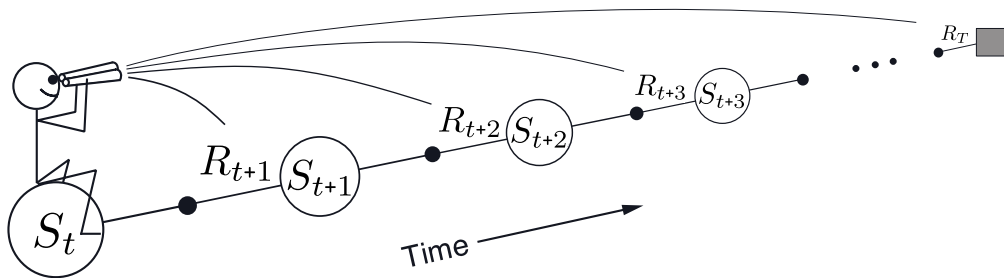
$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

TD(λ) Weighing Function



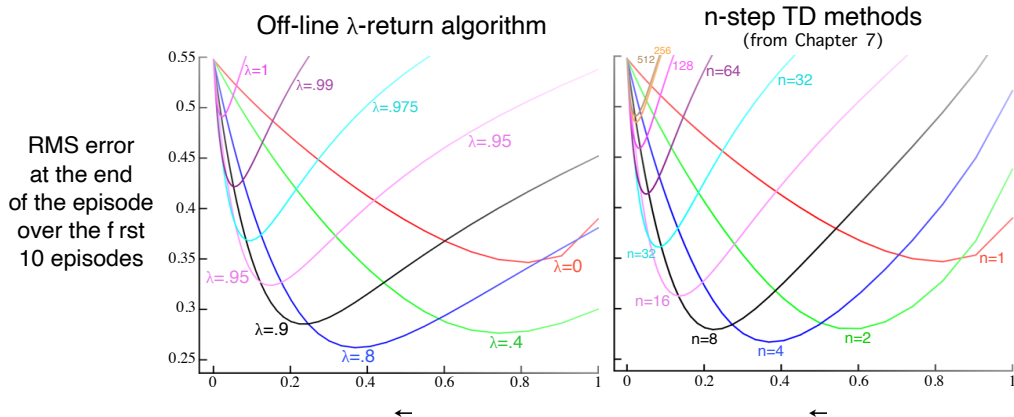
$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)} + \lambda^{T-t-1} G_t$$

Forward-view TD(λ)



- Update value function towards the λ -return
- Forward-view looks into the future to compute G_t^λ
- Like MC, can only be computed from complete episodes

Forward-view TD(λ) on large random walk



Backward-view TD(λ)

- Forward view provides theory
- Backward view provides mechanism
- Update online, every step, from incomplete sequences

Eligibility Traces



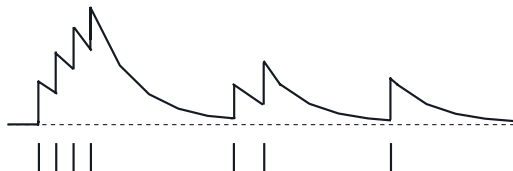
Eligibility Traces



- Credit assignment problem: did bell or light cause shock?
- **Frequency heuristic:** assign credit to most frequent states
- **Recency heuristic:** assign credit to most recent states
- **Eligibility traces** combine both heuristics

$$Z_0(s) = 0$$

$$Z_t(s) = \gamma \lambda Z_{t-1}(s) + 1(S_t = s)$$



accumulating eligibility trace

times of visits to a state

Backward View TD(λ)

- Keep an eligibility trace for every state s
- Update value $V(s)$ for every state s
- In proportion to TD-error δ_t and eligibility trace $Z_t(s)$

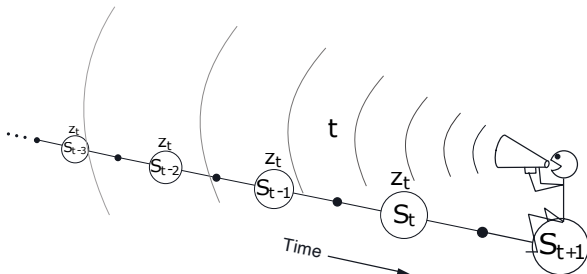
$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

$$V(s) \leftarrow V(s) + \alpha \delta_t Z_t(s)$$

Backward View TD(λ)

- Keep an eligibility trace for every state s
- Update value $V(s)$ for every state s
- In proportion to TD-error δ_t and eligibility trace $Z_t(s)$

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$
$$V(s) \leftarrow V(s) + \alpha \delta_t Z_t(s)$$



TD(λ) Prediction

TD(λ) for estimating $V \approx v_\pi$

Require: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

Require: All store and access operations (for S_t and R_t) can take their index mod $n + 1$

```
1: function lambdaTDPrediction( $\alpha, \pi, \lambda$ )
2:   Initialize  $V(s) \in \mathbb{R}$  arbitrarily for all  $s \in \mathcal{S}^+$ 
3:   loop for each episode
4:     Initialize  $S$ 
5:      $Z(s) \leftarrow 0$  for all  $s \in \mathcal{S}$ 
6:     loop for each step of episode
7:        $a \leftarrow \sim \pi(\cdot \mid S)$ 
8:       Observe and store the next reward as  $R$  and state as  $S'$ 
9:        $\delta \leftarrow R + \gamma V(S') - V(S)$ 
10:       $Z(S) \leftarrow Z(S) + 1$ 
11:      for all  $s \in \mathcal{S}$  do
12:         $V(s) \leftarrow V(s) + \alpha \delta Z(s)$ 
13:         $Z(s) \leftarrow \gamma \lambda Z(s)$ 
14:       $S \leftarrow S'$ 
    until  $S$  is terminal
```

TD(λ) and TD(0)

- When $\lambda = 0$, only current state is updated

$$Z_t(s) = 1(S_t = s)$$

$$V(s) \leftarrow V(s) + \alpha \delta_t Z_t(s)$$

- This is exactly equivalent to TD(0) update

$$V(s) \leftarrow V(s) + \alpha \delta_t$$

TD(λ) and MC

- When $\lambda = 1$, credit is deferred until end of episode
- Consider episodic environments with offline updates
- Over the course of an episode, total update for TD(1) is the same as total update for MC

Theorem

The sum of offline updates is identical for forward-view and backward-view TD(λ)

$$\sum_{t=1}^T \alpha \delta_t Z_t(s) = \sum_{t=1}^T \alpha \left(G_t^\lambda - V(S_t) \right) 1(S_t = s)$$

MC and TD(1)

- Consider an episode where s is visited once at time-step k
- TD(1) eligibility trace discounts time since visit,

$$\begin{aligned} Z_t(s) &= \gamma Z_{t-1}(s) + 1(S_t = s) \\ &= \begin{cases} 0 & \text{if } t < k \\ \gamma^{t-k} & \text{if } t \geq k \end{cases} \end{aligned}$$

- TD(1) updates accumulate error online

$$\sum_{t=1}^T \alpha \delta_t Z_t(s) = \alpha \sum_{t=k}^T \gamma^{t-k} \delta_t = \alpha (G_k - V(S_k))$$

- By end of episode it accumulates total error

$$\delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2} + \cdots + \gamma^{T-1+k} \delta_{T-1}$$

Telescoping TD(1)

When $\lambda = 1$, sum of TD errors telescopes into MC error,

$$\begin{aligned} & \delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2} + \cdots + \gamma^{T-1+k} \delta_{T-1} \\ &= R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \\ &+ \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - \gamma V(S_{t+1}) \\ &+ \gamma^2 R_{t+3} + \gamma^3 V(S_{t+3}) - \gamma^2 V(S_{t+2}) \\ &\vdots \\ &+ \gamma^{T-1-t} R_T + \gamma^{T-t} V(S_T) - \gamma^{T-1-t} V(S_{T-1}) \\ &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^{T-1-t} R_T - V(S_t) \\ &= G_t - V(S_t) \end{aligned}$$

TD(λ) and TD(1)

- TD(1) is roughly equivalent to every-visit Monte-Carlo
- Error is accumulated online, step-by-step
- If value function is only updated offline at end of episode
Then total update is exactly the same as MC

Telescoping TD(λ)

For general λ , TD errors also telescope to λ -error, $G_t^\lambda - V(S_t)$

$$\begin{aligned} G_t^\lambda - V(S_t) &= -V(S_t) && + (1 - \lambda)\lambda^0(R_{t+1} + \gamma V(S_{t+1})) \\ &&& + (1 - \lambda)\lambda^1(R_{t+1} + \gamma R_{t+2} + \gamma V(S_{t+2})) \\ &&& + (1 - \lambda)\lambda^2(R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma V(S_{t+3})) \dots \\ &= -V(S_t) && + (\gamma \lambda)^0(R_{t+1} + \gamma V(S_{t+1}) + \gamma \lambda V(S_{t+1})) \\ &&& + (\gamma \lambda)^1(R_{t+2} + \gamma V(S_{t+2}) + \gamma \lambda V(S_{t+2})) \\ &&& + (\gamma \lambda)^2(R_{t+3} + \gamma V(S_{t+3}) + \gamma \lambda V(S_{t+3})) \dots \\ &= && (\gamma \lambda)^0(R_{t+1} + \gamma V(S_{t+1}) + \gamma V(S_t)) \\ &&& + (\gamma \lambda)^1(R_{t+2} + \gamma V(S_{t+2}) + \gamma V(S_{t+1})) \\ &&& + (\gamma \lambda)^3(R_{t+3} + \gamma V(S_{t+3}) + \gamma V(S_{t+2})) \dots \\ &= \delta_t + && \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 \delta_{t+2} + \dots \end{aligned}$$

Forwards and Backwards TD(λ)

- Consider an episode where s is visited once at time-step k
- TD(λ) eligibility trace discounts time since visit,

$$\begin{aligned} Z_t(s) &= \gamma \lambda Z_{t-1}(s) + 1(S_t = s) \\ &= \begin{cases} 0 & \text{if } t < k \\ (\gamma \lambda)^{t-k} & \text{if } t \geq k \end{cases} \end{aligned}$$

- Backwards TD(λ) updates accumulate error **online**

$$\sum_{t=1}^T \alpha \delta_t Z_t(s) = \alpha \sum_{t=k}^T (\gamma \lambda)^{t-k} \delta_t = \alpha \left(G_t^\lambda - V(S_t) \right)$$

- By end of episode it accumulates total error for λ -return
- For multiple visits to s , $Z_t(s)$ accumulates many errors

Offline Equivalence of Forward and Backward TD

Offline updates

- Updates are accumulated within episode
- but applied in batch at the end of episode

Online Equivalence of Forward and Backward TD

Online updates

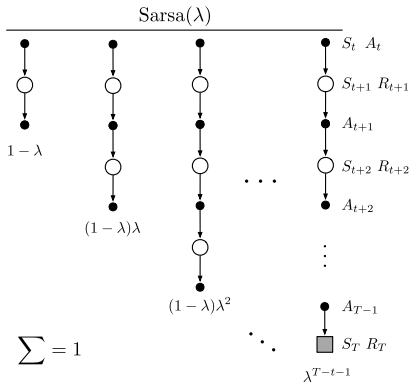
- $TD(\lambda)$ updates are applied online at each step within episode
- Forward and backward-view $TD(\lambda)$ are slightly different
- **NEW:** Exact online $TD(\lambda)$ achieves perfect equivalence
- By using a slightly different form of eligibility trace
Sutton and von Seijen, ICML 2014

Summary of Forward and Backward TD(λ)

Offline updates	$\lambda = 0$	$\lambda \in (0, 1)$	$\lambda = 1$
Backward view	TD(0)	TD(λ)	TD(1)
	$\parallel$	$\parallel$	$\parallel$
Forward view	TD(0)	Forward TD(λ)	MC
Online updates	$\lambda = 0$	$\lambda \in (0, 1)$	$\lambda = 1$
Backward view	TD(0)	TD(λ)	TD(1)
	$\parallel$	$\nparallel$	$\nparallel$
Forward view	TD(0)	Forward TD(λ)	MC
	$\parallel$	$\parallel$	$\parallel$
Exact Online	TD(0)	Exact Online TD(λ)	Exact Online TD(1)

= here indicates equivalence in total update at end of episode

Forward View Sarsa(λ)



- The q^λ return combines n -step Q -returns $q_t^{(n)}$
- Using weight $(1 - \lambda)\lambda^{n-1}$

$$q_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} q_t^{(n)}$$

- Forward-view Sarsa(λ)

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(q_t^\lambda - Q(S_t, A_t) \right)$$

Backward View Sarsa(λ)

- Just like TD(λ), we use eligibility traces in an online algorithm
- But Sarsa(λ) has one eligibility trace for each state-action pair

$$Z_0(s, a) = 0$$

$$Z_t(s, a) = \gamma \lambda Z_{t-1}(s, a) + 1(S_t = s, A_t = a)$$

- $Q(s, a)$ is updated for every state s and action a
- In proportion to TD-error δ_t and eligibility trace $Z_t(s, a)$

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t Z_t(s, a)$$

Sarsa(λ) Algorithm

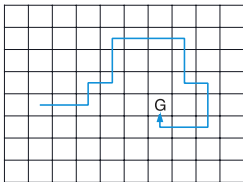
Sarsa(λ) for estimating $Q \approx q_*$ or q_π

Require: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

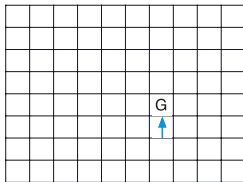
```
1: function lambdaSarsaControl( $\alpha, \epsilon, \lambda$ )
2:   Initialize  $Q(s, a) \in \mathbb{R}$ , for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$  arbitrarily except that  $Q(\text{terminal}, \cdot) = 0$ 
3:   loop for each episode
4:      $Z(s, a) = 0$  for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$ 
5:     Initialize  $S, A$ 
6:     loop for each step of the episode
7:       Take action  $A$ , observe  $R, S'$ 
8:       Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.  $\epsilon$ -greedy)
9:        $\delta \leftarrow R + \gamma Q(S', A') - Q(S, A)$ 
10:       $Z(S, A) \leftarrow Z(S, A) + 1$ 
11:      for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$  do
12:         $Q(s, a) \leftarrow Q(s, a) + \alpha \delta Z(s, a)$ 
13:         $Z(s, a) \leftarrow \gamma \lambda Z(s, a)$ 
14:       $S \leftarrow S'; A \leftarrow A'$ 
    until  $S$  is terminal
```

Sarsa(λ) Gridworld Example

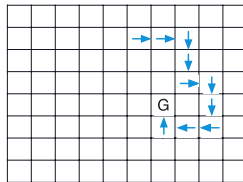
Path taken



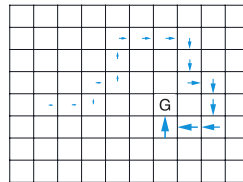
Action values increased
by one-step Sarsa



Action values increased
by 10-step Sarsa



Action values increased
by Sarsa(λ) with $\lambda=0.9$



Outline

- 1 Recap
- 2 *n*-step Bootstrapping
 - n*-step TD Prediction
 - n*-step Sarsa
- 3 Eligibility Traces and λ Algorithms
 - TD(λ)
 - Sarsa(λ)
- 4 From DP to MC and TD

Revisiting the Bellman Equations

Sarsa:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma - Q(S_t, A_t))$$

Revisiting the Bellman Equations

Sarsa:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

$$q_\pi(s, a) = r + \sum_{s'} \mathcal{P}_{ss'}^{a'} \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a')$$

Revisiting the Bellman Equations

Sarsa:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (\textcolor{red}{R}_{t+1} + \textcolor{brown}{\gamma} Q(\textcolor{red}{S}_{t+1}, \textcolor{red}{A}_{t+1}) - Q(S_t, A_t))$$

$$q_\pi(s, a) = \textcolor{red}{r} + \sum_{s'} \mathcal{P}_{ss'}^{a'} \textcolor{brown}{\gamma} \sum_{a'} \pi(a' | s') q_\pi(\textcolor{red}{s}', \textcolor{red}{a}')$$

Revisiting the Bellman Equations

Sarsa:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

$$q_\pi(s, a) = r + \sum_{s'} \mathcal{P}_{ss'}^{a'} \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a')$$

Q-learning:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right)$$

Revisiting the Bellman Equations

Sarsa:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$
$$q_\pi(s, a) = r + \sum_{s'} \mathcal{P}_{ss'}^{a'} \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a')$$

Q-learning:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right)$$
$$q_\pi(s, a) = r + \gamma \max_{a'} \sum_{s'} \mathcal{P}_{ss'}^{a'} q_\pi(s', a')$$

Revisiting the Bellman Equations

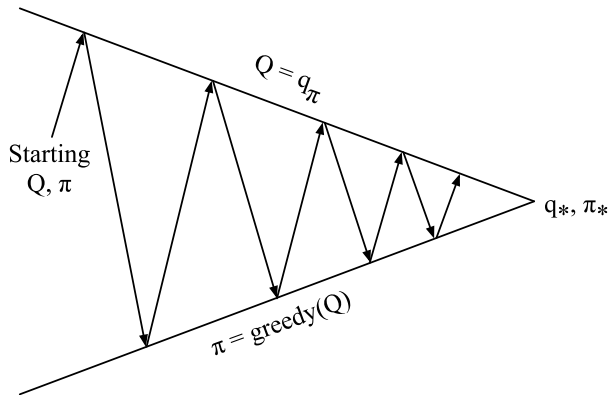
Sarsa:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$
$$q_\pi(s, a) = r + \sum_{s'} \mathcal{P}_{ss'}^{a'} \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a')$$

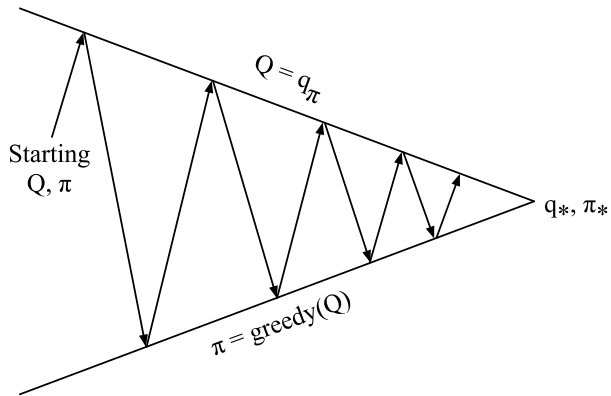
Q-learning:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t))$$
$$q_\pi(s, a) = r + \gamma \max_{a'} \sum_{s'} \mathcal{P}_{ss'}^{a'} q_\pi(s', a')$$

Connections to Dynamic Programming

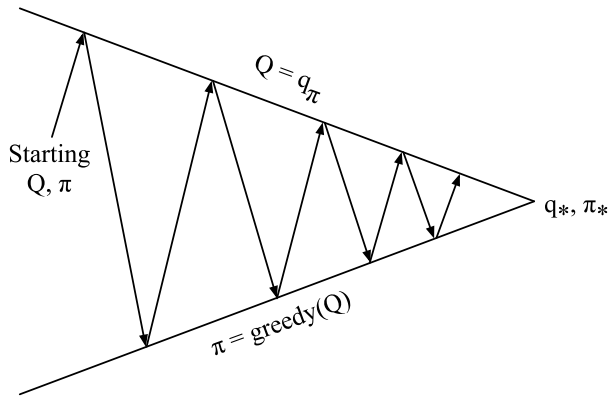


Connections to Dynamic Programming



Sarsa - Policy Iteration

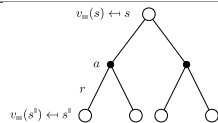
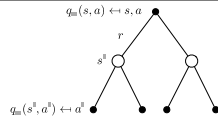
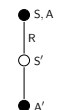
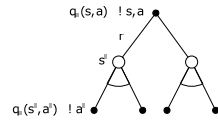
Connections to Dynamic Programming



Sarsa - Policy Iteration

Q-learning - Value Iteration

Relationship Between DP and TD

	Full Backup (DP)	Sample Backup (TD)
Bellman Expectation Equation for $v_{\pi}(s)$	 Iterative Policy Evaluation	 TD Learning
Bellman Expectation Equation for $q_{\pi}(s, a)$	 Q-Policy Iteration	 Sarsa
Bellman Optimality Equation for $q_*(s, a)$	 Q-Value Iteration	 Q-Learning

Relationship Between DP and TD (2)

Full Backup (DP)	Sample Backup (TD)
Iterative Policy Evaluation	TD Learning (TD)
$V(s) \leftarrow \mathbb{E} [R + \gamma V(S') \mid s]$	$V(S) \stackrel{\alpha}{\leftarrow} R + \gamma V(S')$
Q-Policy Iteration	Sarsa
$Q(s, a) \leftarrow \mathbb{E} [R + \gamma Q(S', A') \mid s, a]$	$Q(S, A) \stackrel{\alpha}{\leftarrow} R + \gamma Q(S', A')$
Q-Value Iteration	Q-Learning
$Q(s, a) \leftarrow \mathbb{E} \left[R + \gamma \max_{a'} Q(S', a') \mid s, a \right]$	$Q(S, A) \stackrel{\alpha}{\leftarrow} R + \gamma \max_{a'} Q(S', a')$

where $x \stackrel{\alpha}{\leftarrow} y \equiv x \leftarrow x + \alpha(y - x)$

Applicability of λ updates

We can use λ updates with other RL-algorithms we see in this course:

- Planning and Learning (Dyna-Q, Lecture 10)
 λ updates can average experience-based updates with DP updates
- Function approximation (Lecture 11)
 λ updates can average updates to approximation functions (e.g., linear, ANN, ...)

Lecture Summary

- Unified view of MC and TD using variable-step bootstrapping:
 - n-step returns for TD and Sarsa
 - Eligibility traces and λ -returns
 - TD(λ) and Sarsa(λ)
- Review of the relationship between RL and DP